JAM: Action-Grounded World Pretraining

Lixuan He

September 2026

Abstract

Video pretraining provides a prior over how scenes evolve. Learning to act also requires a representation of how control relates to that evolution. We study whether action supervision can shape a pretrained world representation through the computation used to generate future observations. JAM, Joint Action–World Modeling, jointly generates future video, motor actions, and visual action consequences. A video backbone and an agent transformer exchange attention keys and values at every layer, allowing action prediction to update the world stream. Visual consequences describe image-space displacement and observability, extending the same training objective to human demonstrations with tracked motion. Independent corruption levels support learning from modalities with different levels of uncertainty. The available training record shows decreasing world, action, and consequence objectives in a completed task adaptation run, while foundation pretraining continues on robot and human data. Controlled comparisons are organized around the effects of information exchange, gradient routing, and consequence supervision on representation quality and transfer. This formulation makes the influence of action learning on world pretraining a concrete, testable property of the model.

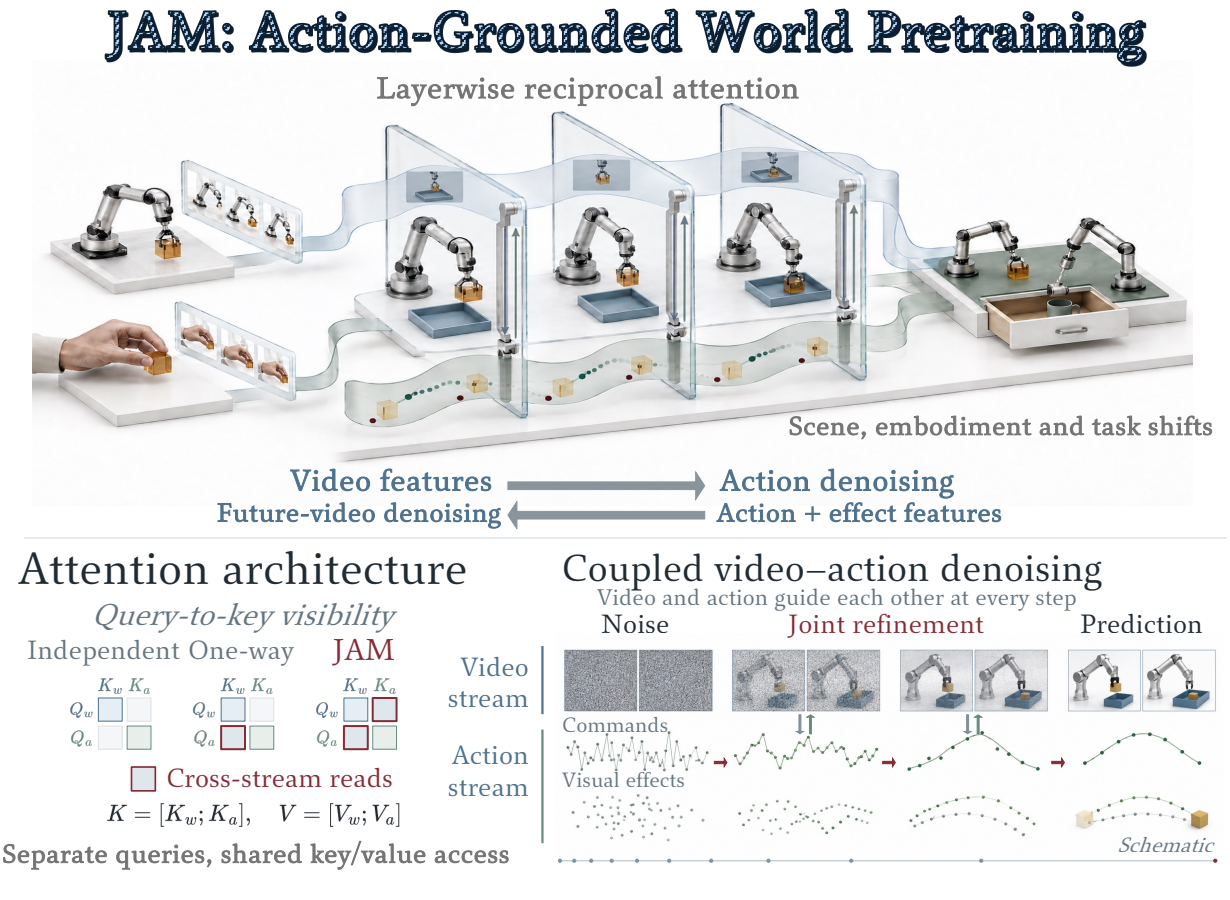

Figure 1: Reciprocal action and world learning. JAM exchanges video and action information at every layer. The lower panels show attention visibility and coupled denoising of future video, commands, and visual consequences. Images are schematic. The illustrated scene, embodiment, and task shifts denote evaluation settings.

1 Introduction

Video models learn regularities in appearance, motion, and scene evolution from visual sequences. These regularities provide useful starting points for embodied learning: a robot must anticipate how objects move, how contact changes a scene, and which futures are consistent with its observations. Recent work transfers video generation to control by extracting actions from imagined trajectories or adapting video models to generate actions directly (Du et al., 2023; Kim et al., 2026). This progress motivates a question about the pretraining process itself: *how should learning to act influence the representation used to predict the world?*

A manipulation video records an interaction through its visible outcome. Robot demonstrations additionally record the commands that accompanied it. These two descriptions constrain each other. The same scene can evolve differently under different commands, and a desired change in the scene can admit several feasible actions. A representation for embodied interaction should preserve these dependencies. Action grounding, in this work, refers to the extent to which a world representation retains information useful for relating commands to their visual consequences. We examine this property through prediction, controlled changes to the inputs, and transfer to new conditions.

We introduce JAM, Joint Action–World Modeling, as a way to expose the world representation to action supervision during pretraining. JAM generates actions and future observations within a shared denoising computation. A pretrained video stream and an agent stream exchange information throughout their depth. Their attention connections also carry gradients, so learning to predict actions can update the features used for world prediction. The central hypothesis is that this reciprocal computation encourages a representation of interaction that remains useful when the environment or task changes. The model makes this hypothesis experimentally accessible through separate controls over information visibility and gradient flow.

The relationship between commands and visible change also offers a route to human video. A motor command is specific to an embodiment and its controller. A visual action consequence describes where tracked points move in the image and whether they remain observable. JAM represents these consequences as generated tokens within the agent stream. Robot and human demonstrations can therefore supervise a common motion interface while retaining their own available annotations. This construction also provides a diagnostic: consequence prediction tests which aspects of interaction are accessible from the learned world representation.

Our study follows the influence of action supervision from the training computation to its downstream use. We first formulate the joint prediction problem and explain how the streams interact. We then describe pretraining with heterogeneous supervision and adaptation for control. The experiments connect optimization evidence to controlled tests of coupling, data composition, and transfer. Together, this organization asks when learning actions and visual consequences changes what a world model represents, and how that change can be measured.

2 Related Work

Video priors for embodied learning. Video generation learns temporal structure from visual data (Wan Team, 2025). UniPi uses generated videos as plans from which actions are recovered (Du et al., 2023). Cosmos Policy incorporates actions, future observations, and values into a pretrained video model through latent frames (Kim et al., 2026). These approaches establish practical routes from generative priors to control. JAM focuses on how action learning influences the video representation during embodied pretraining, with explicit controls over the connections carrying information and gradients.

Joint action and world prediction. Unified World Models (UWM) jointly diffuse actions and observations, sample their noise levels independently, and obtain policies and dynamics models through conditioning (Zhu et al., 2025). JAM builds on this joint-distribution perspective and uses rectified-flow training (Lipman et al., 2023; Liu et al., 2023b). Its study centers on a pretrained video stream coupled throughout its depth to an agent stream. Separating visibility from gradient routing permits comparisons between reading world features and updating those features through action supervision. Conditional sampling provides an additional use of the learned distribution; the representation question motivates the experimental design.

Action representations across embodiments. Robot policy pretraining aligns observations and language with commands across datasets and platforms (Kim et al., 2024; Khazatsky et al., 2024; Walke et al., 2023). Visual trajectories offer another interface to interaction: Any-point Trajectory Modeling learns motion representations for policies, and Track2Act converts predicted point tracks into executable motion (Wen et al., 2024; Bharadhwaj et al., 2024). EgoDex supplies human manipulation video with tracked hand poses (Hoque et al., 2025). JAM incorporates image-space consequences into its joint generative objective, alongside motor commands where available. The corresponding experiments examine command–consequence complementarity and transfer across embodiments under explicitly matched observation and annotation conditions.

3 Joint Action–World Modeling

The design starts from a simple requirement: action prediction and world prediction should be able to use and update each other's representations. A joint objective specifies which variables are generated. The attention structure determines how learning those variables interacts.

3.1 The prediction problem

Let o_0 be the current observation, ℓ a language instruction, and s the available proprioceptive state. We write $h = (o_0, \ell, s)$ for these conditions, with a learned null token for missing state. Let z denote encoded future observations and a a chunk of future motor actions. JAM models $p_\theta(z, a \mid h)$. When visual consequence labels c are available, the same computation models $p_\theta(z, a, c \mid h)$. A fixed video autoencoder maps observations to latents; its decoder maps generated latents back to images. Action and consequence tokens preserve their respective control and image-space meanings.

For each generated modality $m \in \{w, a, c\}$, let $x^w = z$, $x^a = a$, and $x^c = c$. We draw independent standard Gaussian noise ϵ^m and a corruption level τ_m between clean and fully noisy. Rectified-flow training constructs

$$x^m_{\tau_m} = (1 - \tau_m)x^m + \tau_m \epsilon^m, \qquad u^m = \epsilon^m - x^m. \tag{1}$$

Here u^m is the target velocity along the interpolation from data to noise. The model predicts all available velocities from the corrupted modalities and the clean conditions:

$$(v^w_\theta, v^a_\theta, v^c_\theta) = F_\theta(z_{\tau_w}, a_{\tau_a}, c_{\tau_c}; h, \tau_w, \tau_a, \tau_c). \tag{2}$$

The current configuration samples consequence corruption independently of action corruption. This keeps the confidence of the consequence input distinct from that of the motor input. The formulation follows independent-modality corruption in UWM (Zhu et al., 2025); the coupling described below specifies how the pretrained representations participate.

3.2 Reciprocal computation across streams

The world stream retains a pretrained video transformer's latent embeddings, blocks, and output projection. The agent stream contains a state token followed by action and consequence tokens. It uses separate input projections for each token type and noise-dependent adaptive normalization. Both streams receive the language condition. Their residual widths may differ; their attention projections share a common head geometry.

At each coupled layer, each stream forms its own queries, keys, and values. World queries attend to world and agent keys; agent queries attend to agent and world keys. In schematic form, for stream m and the other stream $\bar{m}$,

$$H'_m = H_m + P_m \ \mathrm{Attn}\big(Q_m, [K_m; K_{\bar{m}}], [V_m; V_{\bar{m}}]\big), \tag{3}$$

where H_m is the layer input, P_m maps attention outputs to its residual width, and brackets denote token concatenation. Each block then applies language cross-attention and a feed-forward update. Consequence

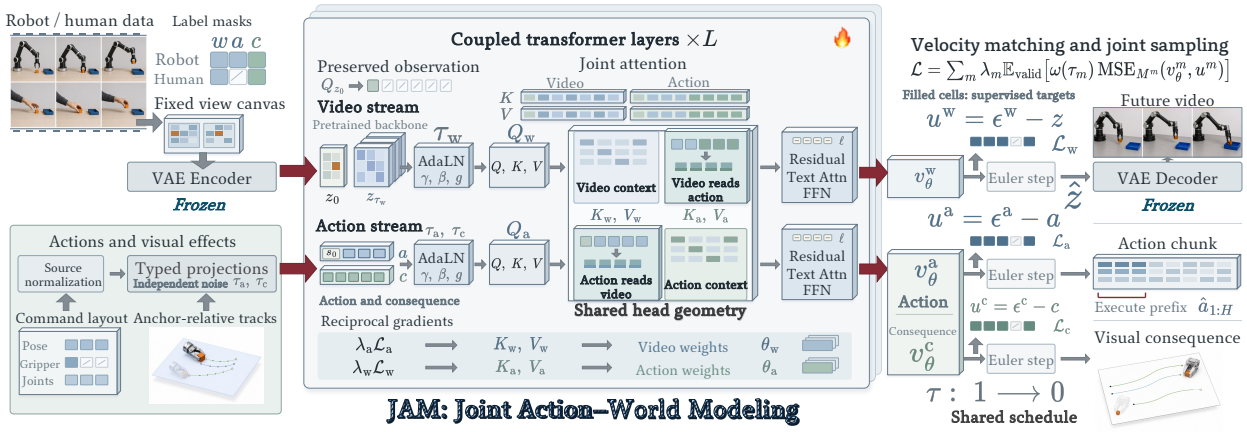

Figure 2: How action learning enters world pretraining. Source masks select available video, action, and consequence targets. Encoded future observations and projected action and consequence tokens receive independent corruption. A shared head geometry supports cross-stream attention, which carries information and reciprocal gradients through the coupled streams, while the current observation remains a clean prefix. Masked velocity matching trains their readouts; joint Euler integration produces future video, an action chunk, and visual consequences. VAE denotes variational autoencoder, AdaLN adaptive layer normalization, FFN feed-forward network, and MSE mean squared error. Interaction images are schematic.

tokens belong to the agent computation and participate in the same exchange. Figure 2 shows the complete training flow.

The default coupling is bidirectional at every layer, with the exchanged tensors attached to the computation graph. Thus action loss has a route into world parameters through the world keys and values read by the agent; world loss has a reciprocal route into agent parameters. These routes establish a mechanism for action supervision to influence the world representation. Their value for transfer is an empirical question. Visibility masks and gradient stops expose the mechanism to controlled comparison.

The current observation forms a clean prefix in the world sequence. Prefix queries attend only to the clean prefix, while generated world tokens can read the prefix and the agent stream. This preserves an observation context throughout joint prediction. The state token is a clean input to the agent stream and is excluded from the generation loss.

3.3 Learning under partial supervision

Each example carries coordinate masks M^m for the labels it supplies. We use a masked mean squared error, MSE_{M^m}, averaged over valid coordinates within that example. Let $\mathcal{B}_m$ be the batch examples with at least one supervised coordinate for modality m. The implemented objective is

$$\mathcal{L} = \sum_{m \in \{w,a,c\}} \lambda_m \frac{1}{|\mathcal{B}_m|} \sum_{i \in \mathcal{B}_m} \omega(\tau_{m,i})\ \mathrm{MSE}_{M_i^m}\left(v_{\theta,i}^m, u_i^m\right), \tag{4}$$

where λ_m controls each modality's contribution and ω weights its noise level. An empty supervised set contributes zero. World loss covers future latents; the clean prefix is excluded. Missing coordinates are masked after corruption as well as in the loss. Robot batches supervise actions and video, while human batches supervise video and the available consequences. Consequence annotations may also accompany robot data. Appendix A gives the implemented sampling distribution and normalization rules.

4 Action-Grounded World Pretraining

Joint prediction provides the learning mechanism. Its use across demonstrations depends on preserving what each source actually observes and controls. We therefore align the meaning of the inputs before combining their supervision.

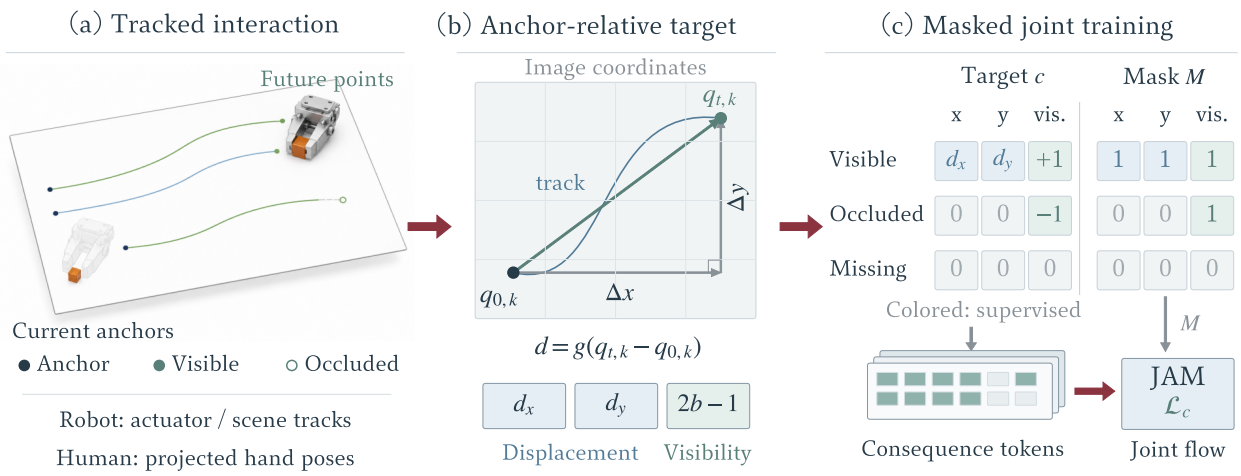

Figure 3: What consequence supervision represents. (a) Tracks connect current anchors to future image positions. Robot labels can combine actuator and scene tracks; the active human adapter projects supplied hand poses. (b) The target contains gain-scaled displacement d and observability. (c) Binary visibility examples distinguish visible points, occluded points, and missing slots. An occluded point retains visibility supervision; its stored zero displacement is masked. A missing slot is fully masked. Consequence tokens enter JAM's joint flow objective, with mask M selecting supervised coordinates. The interaction and mask cases are explanatory schematics.

4.1 Commands and their visual consequences

A source adapter maps motor commands into a typed layout for end-effector pose, gripper aperture, joints, and additional embodiment-specific coordinates. The adapter also records which coordinates exist. Source-specific normalization is applied after this mapping. Rotation coordinates retain their geometric convention, and gripper aperture follows a shared closed-to-open convention. This interface preserves physical meaning when sources populate different subsets of the layout.

Visual action consequences describe displacement from observation-time anchors. For keypoint k at future time t, let $q_{t,k}$ be its image-normalized position, $q_{0,k}$ its anchor, and $b_{t,k}$ its observability. The target is

$$c_{t,k} = \left[g(q_{t,k} - q_{0,k}),\ 2b_{t,k} - 1 \right], \tag{5}$$

where g is a fixed displacement gain. Tracked actuator points use their observation-time position as the anchor; scene probes use their query location in that frame. Masks distinguish missing slots from observed low visibility. The resulting sequence is projected into consequence tokens and trained by the same velocity objective as actions and video.

Figure 3 illustrates this interface. Point trackers provide one route to such labels (Karaev et al., 2024). The active human-data adapter instead projects supplied hand poses through the recorded camera calibration and combines their confidence with visibility (Hoque et al., 2025). These demonstrations supply tracked motion while motor commands remain unobserved. Consequences express visible motion in a common coordinate format; camera motion, projection, and embodiment still affect their distribution. Cross-embodiment invariance is therefore evaluated through held-out transfer.

4.2 Pretraining from heterogeneous demonstrations

Each training window contains an instruction, a current observation, future frames, and the available state, action, and consequence annotations. Camera views occupy fixed positions in an image canvas, which is encoded by the frozen video autoencoder. A frozen text encoder supplies language embeddings. Learned input projections route control and consequence coordinates into the agent stream. The pretrained world transformer and the new agent transformer are optimized together.

We sample sources using a temperature applied to their numbers of valid training windows. Each micro-batch contains a single source so its camera geometry and annotation pattern remain consistent. Robot

and human batches enter the same optimization stage. Human data thus contributes a world objective and a consequence objective while robot data anchors the command interface. The current recipe and training budget appear in Section 5; larger acquired corpora enter the accounting when their caches are incorporated into a run.

Increasing data, updates, and model capacity answers different scaling questions. The study varies each separately: subsets of a fixed corpus test data diversity, checkpoint milestones test training duration, and agent capacity tests the cost of the learned control interface. The key comparison preserves robot exposure when adding human data, followed by a compute-matched comparison. This separates an additional supervision source from an additional optimization budget.

4.3 Adaptation and conditional use

Downstream adaptation retains the joint objective and replaces the data mixture with demonstrations for the target environment. At execution time, JAM starts the generated streams from noise, integrates predicted velocities toward clean samples, and executes a prefix of the action chunk before observing again. The world decoder and consequence readout make the jointly sampled future available for diagnostics.

The joint formulation also supports action-conditioned world prediction and inverse action prediction from future observations. Independent corruption varies uncertainty in each input. Each sampler specifies supplied variables, latent variables, and their integration schedules. The current policy uses a shared decreasing schedule; conditional comparisons await sampler validation.

5 Experiments

5.1 Experimental setup

JAM combines robot demonstrations with human video in embodied pretraining. The current mixture uses DROID (Khazatsky et al., 2024), BridgeData V2 (Walke et al., 2023), EgoDex (Hoque et al., 2025), Ego4D (Grauman et al., 2022), and the Franka, UR5e, and AgileX portions of RoboMIND (Wu et al., 2025). Appendix B gives source counts and sampling probabilities. The 6.02-billion-parameter model combines public video initialization with a newly initialized agent stream, trained on eight H200 accelerators with global batch 512. Episode-level holdouts separate training windows from open-loop checks; benchmark adaptation uses designated training splits, with LIBERO-Plus and LIBERO-PRO reserved for evaluation.

The evidence snapshot contains foundation optimization through update 10,501, a completed 20,000-update LIBERO adaptation, and frozen-world diagnostics at update 5,100. The benchmark tables retain source-reported baseline values and reserve blue rows for JAM evaluation. Paired comparisons fix demonstrations, adaptation budgets, and environment resets. Success measures closed-loop control; frozen-feature readouts and input sensitivity assess representation content. Controlled pretraining and scaling displays retain target markers while measurements are pending. Appendix C specifies probe sampling and uncertainty.

5.2 Benchmark control and generalization

Table 1 resolves robustness across seven perturbation axes.

LIBERO-Plus								
Model	Camera	Robot	Language	Light	Backgr.	Noise	Layout	Mean
π_0	13.8	6.0	58.8	85.0	81.4	79.0	68.9	53.6
OpenVLA-OFT	56.4	31.9	79.5	88.7	93.3	75.8	74.2	69.6
StarVLA	52.5	49.8	88.5	95.7	95.7	73.0	76.9	74.1
ABot-M0	60.4	67.9	86.4	96.2	91.6	86.4	82.6	80.5
$\pi_{0.5}$	78.4	73.6	80.8	96.2	94.1	89.0	84.5	84.4
ACoT-VLA	72.6	82.6	87.5	97.7	96.5	87.8	88.1	86.6
Qwen-RobotManip	**87.2**	75.5	85.6	96.6	**97.7**	**97.7**	87.3	**89.0**
Fast-WAM	16.4	44.5	68.9	78.2	53.7	37.7	60.7	51.5
Being-H0.7	82.0	59.0	82.8	97.8	90.0	93.5	**88.5**	82.1
Cosmos Policy	75.8	63.3	81.7	96.5	88.9	92.7	82.2	82.2
ImageWAM	80.8	50.3	**91.4**	**98.1**	85.5	93.8	80.5	83.1
ABot-M0.5	70.5	**87.4**	88.6	94.0	89.7	75.5	85.2	83.4
R1	33.8	76.1	88.0	97.0	87.1	39.8	77.5	69.2
JAM	Awaiting evaluation							

Table 1: LIBERO-Plus robustness. Success percentages across seven perturbation axes; the mean retains the source aggregation. Source: external reference R1. Bold marks the highest reported external value per column; NR means unreported. The blue JAM row awaits a completed evaluation.

Table 2 summarizes robustness within four task suites.

LIBERO-PRO					
Model	Goal	Spatial	Long	Object	Total
OpenVLA	58.4	56.6	52.4	39.2	52.0
π_0	45.2	50.4	37.6	42.6	44.0
$\pi_{0.5}$	55.6	52.2	48.0	57.0	53.0
MolmoAct*	38.3	44.5	33.5	48.5	41.0
NORA*	36.5	45.8	30.0	44.5	40.0
X-VLA*	44.0	48.3	41.8	49.3	46.0
JAM	Awaiting evaluation				

Table 2: LIBERO-PRO suite performance. Suite values are macro-averages of published perturbation percentages; Total is copied from the benchmark-maintainer leaderboard. *Averages cover four reported perturbations, with environment tests unreported. The blue JAM row awaits evaluation.

Table 3 summarizes task success, progress, and intention on VLABench.

VLABench			
Model	SR	PS	IS
π_0	29.4	44.1	55.0
LoHo-Manip	39.0	NR	NR
ACoT-VLA	NR	47.4	63.5
$\pi_{0.5}$	48.1	62.3	64.9
ERVLA	53.2	65.9	70.4
Xiaomi-Robotics-1	**59.1**	**70.3**	69.9
Bridge-WA	52.8	64.0	**71.2**
R1	58.9	67.2	63.5
JAM	Awaiting evaluation		

Table 3: VLABench overall performance. SR, PS, and IS denote task success, progress score, and intention score. Values retain the source-reported overall percentage scale. Source: external reference R1. Bold marks the highest reported external value per column; NR means unreported. The blue JAM row awaits a completed evaluation.

Table 4 compares bimanual control on RoboTwin2.0-Full and RoboDojo.

A. RoboTwin2.0-Full

Model	Clean	Rand.	Mean
X-VLA	72.80	72.84	72.82
$\pi_{0.5}$	82.70	76.80	79.75
ABot-M0	86.06	85.08	85.57
Qwen-VLA	86.10	87.20	86.65
StarVLA	88.18	88.32	88.25
Galaxea G0.5	93.70	92.80	93.25
Qwen-RobotManip	93.70	94.00	93.85
Motus	88.66	87.02	87.84
Fast-WAM	91.90	91.80	91.85
LingBot-VA	92.93	91.55	92.24
ImageWAM	93.20	93.56	93.38
LingBot-VA 2.0	93.80	93.40	93.60
ABot-M0.5	**94.00**	**94.20**	**94.10**
R1	93.74	93.46	93.60
JAM	Awaiting evaluation		

B. RoboDojo

Model	SR	Score
StarVLA-α	3.24	6.40
X-VLA	6.52	10.13
$\pi_{0.5}$	6.91	11.41
Spatial Forcing	8.04	12.38
Hy-Embodied-0.5-VLA	8.80	13.07
Xiaomi-Robotics-1	13.93	20.07
Galaxea G0.5	14.88	20.23
DM0.5	**19.34**	**24.90**
Fast-WAM	2.03	3.48
AHA-WAM	2.39	4.82
GigaWorld-Policy	3.27	6.20
X-WAM	3.83	7.69
R1	11.92	17.18
JAM	Awaiting evaluation	

Table 4: Bimanual manipulation. RoboTwin2.0-Full reports clean and randomized success; RoboDojo reports overall success rate (SR) and task score. Source: external reference R1. Bold marks column maxima; blue rows await JAM evaluation.

Table 5 compares mobile manipulation on RoboCasa365 and EBench.

A. RoboCasa365

Model	Atomic	Seen	Unseen	Mean
Diffusion Policy	15.7	0.2	1.3	6.1
π_0	36.3	5.2	0.7	15.0
$\pi_{0.5}$	39.6	7.1	1.2	16.9
GR00T-N1.5	50.7	14.8	2.7	23.9
Qwen-RobotManip	68.6	20.1	14.9	35.9
RLDX-1	67.6	27.9	8.5	36.0
Xiaomi-Robotics-1	**80.2**	**57.1**	**32.1**	**57.4**
GigaWorld-Policy	44.4	11.8	2.9	20.7
ABot-M0.5	75.9	38.3	2.7	40.4
R1	69.7	32.1	8.9	38.2
JAM	Awaiting evaluation			

B. EBench

Model	SR	Score
StarVLA-OFT	0.0	0.2
π_0	23.6	37.0
X-VLA	23.7	36.0
InternVLA-A1	23.9	36.0
$\pi_{0.5}$	27.1	41.0
GigaBrain-0.7	33.3	46.0
Qwen-RobotManip	45.6	60.0
Fast-WAM	4.7	7.6
R1	**49.4**	**64.7**
JAM	Awaiting evaluation	

Table 5: Mobile manipulation. RoboCasa365 separates atomic skills from seen and unseen compositions. EBench reports overall success rate (SR) and task score. Source: external reference R1. Bold marks column maxima; blue rows await JAM evaluation.

5.3 Joint optimization

Figure 4 records the joint objectives during foundation pretraining and task adaptation.

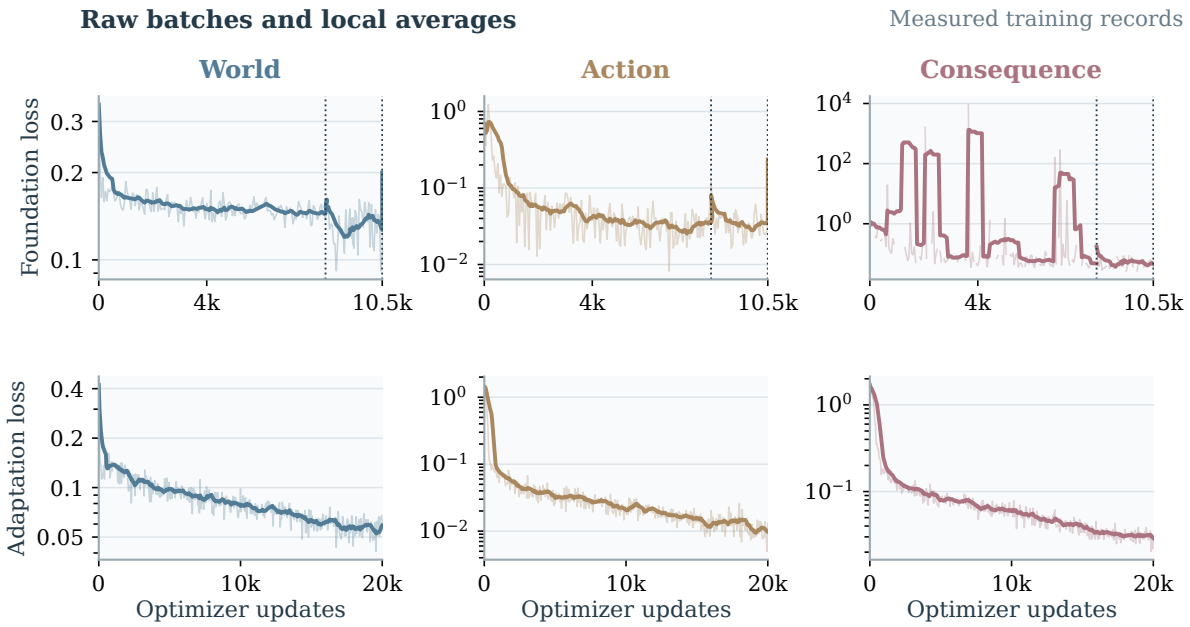

Figure 4: Joint optimization of world, action, and consequence prediction. Foundation pretraining (top) and completed LIBERO adaptation (bottom). Thin traces retain logged batches; thick traces average up to 11 records with available supervision. Each objective uses a logarithmic scale. Vertical rules mark configuration boundaries, where smoothing restarts. The last foundation segment contains one logged update. Appendix A records the changes at each boundary.

5.4 Representation and reciprocal computation

Figure 5 compares motor-action and visual-consequence readout from JAM world features with video and random initialization.

Table 6 separates the value of embodied pretraining from the information and gradient paths within joint computation.

A. Initialization at matched adaptation budget

Initialization	LIBERO	Plus
Without embodied pretraining	97.2^{T}	53.8^{T}
JAM pretraining	99.4^{T}	90.2^{T}

B. Information exchange and gradient routing

Variant	W reads A	A reads W	Gradient	LIBERO	Plus	A probe	C error
World only	Off	Off	Off	94.0^{T}	42.0^{T}	0.30^{T}	0.18^{T}
Action only	Off	Off	Off	95.0^{T}	45.0^{T}	0.32^{T}	0.17^{T}
Independent	Off	Off	Off	96.1^{T}	47.6^{T}	0.38^{T}	0.15^{T}
Auxiliary	Off	On	Off	96.8^{T}	49.4^{T}	0.40^{T}	0.12^{T}
One-way	Off	On	On	97.0^{T}	51.7^{T}	0.46^{T}	0.10^{T}
JAM joint	On	On	On	97.2^{T}	53.8^{T}	0.52^{T}	0.08^{T}

Table 6: Which part of pretraining changes the representation? Panel A isolates embodied pretraining from task adaptation. Panel B holds source exposure and clean conditions fixed. W and A denote world and action; Gradient denotes action-loss access to world parameters. A probe reports R-squared; C error is consequence average displacement error (ADE). All entries in this controlled study await measurement. Superscript T marks layout targets awaiting measurement. Pale blue identifies JAM rows.

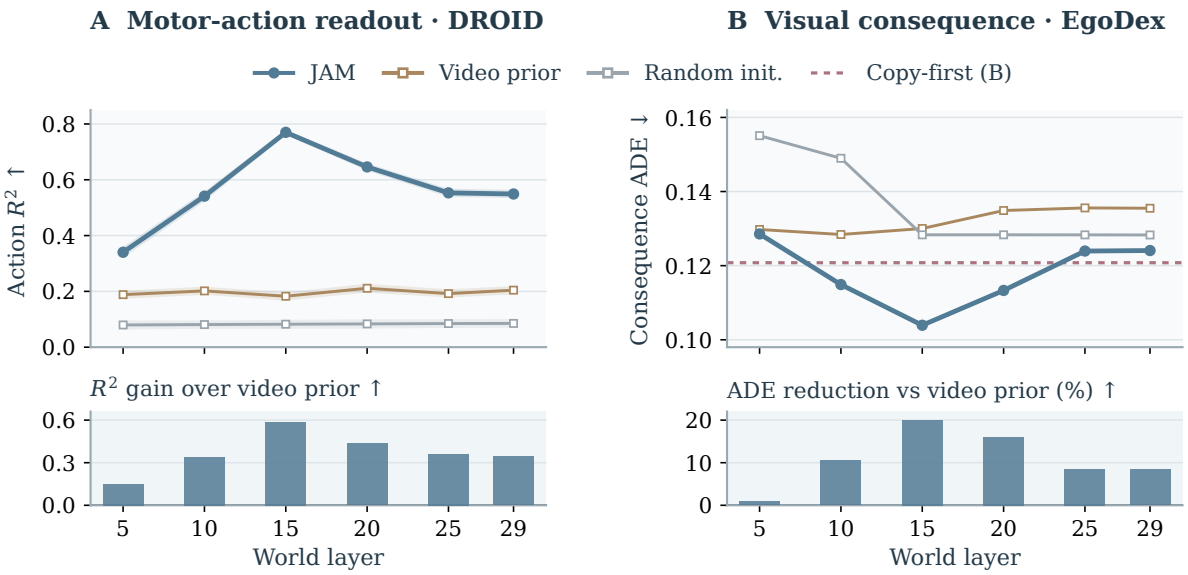

Figure 5: Reading actions and visual consequences from world features. Frozen-world ridge probes compare JAM at update 5,100 with video and random initialization at every sampled layer. Top: action R-squared (higher is better) and consequence average displacement error, ADE (lower is better). Bottom: JAM's gain over the video prior, shown as point estimates. The copy-first reference remains visible in B. Action bands use 1,000 window-bootstrap resamples. Future observations are partly visible; the 2,800/1,200 probe-training/test windows can share episodes (Appendix C). Action-input sensitivity is evaluated separately in Figure 7, where all paired intervals include zero.

5.5 Data, capacity, and supervision

Table 7 and Figure 6 organize matched-budget comparisons of supervision, unique data, duration, and capacity.

A. Which supervision source contributes?						
Mixture	Robot budget	Human	Human C	LIBERO	Plus	C error
Robot	Matched	Off	Off	98.4^{T}	74.2^{T}	0.12^{T}
+ human video	Matched	On	Off	98.9^{T}	82.6^{T}	0.10^{T}
JAM mixture	Matched	On	On	99.4^{T}	90.2^{T}	0.08^{T}

B. Capacity and supervision density				
Axis	Setting	Corpus	Updates	Plus
Agent width	768	Full	60k	85.0^{T}
Agent width	1024	Full	60k	90.2^{T}
Agent width	1280	Full	60k	91.0^{T}
Action labels	25%	Full	60k	82.0^{T}
Action labels	50%	Full	60k	86.0^{T}
Action labels	100%	Full	60k	90.2^{T}

Table 7: Which data and scale choices support action grounding? Panel A matches robot exposure; Panel B fixes the backbone, corpus, and update budget. Success is in percent and C error is image-normalized ADE. Blue rows identify reference settings. Superscript T marks layout targets awaiting measurement. Pale blue identifies JAM rows.

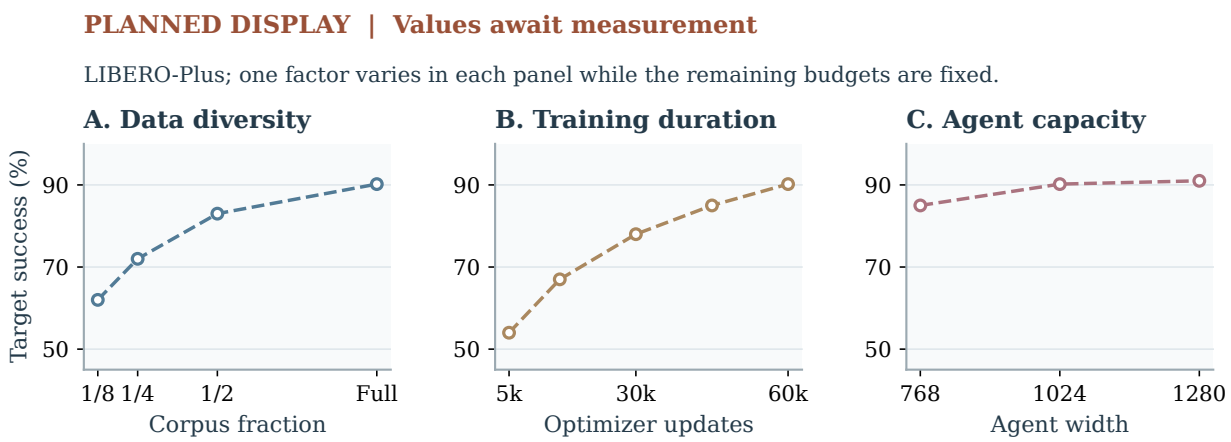

Figure 6: How are data and compute effects separated? Planned LIBERO-Plus displays vary unique data, duration, or agent width at otherwise fixed budgets. All points are PLANNED-VALUE, awaiting measurement; the full-corpus reference uses 60,000 updates.

6 Conclusion

JAM places action prediction inside the computation used to learn world dynamics. Its coupled streams give action supervision a direct gradient route into a pretrained video representation, while visual consequences provide a motion target shared by robot and human demonstrations. The completed adaptation record shows that the implementation can optimize world, action, and consequence prediction together. Foundation training and the controlled evaluation address the remaining scientific question: when does this coupling produce a representation that transfers to new interactions? The present evidence supports the training formulation and motivates measuring representation change separately from optimization progress.

References

Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting Point Tracks from Internet Videos enables Generalizable Robot Manipulation. In *European Conference on Computer Vision*, 2024. URL https://arxiv.org/abs/2405.01527.

Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, et al. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. *arXiv preprint arXiv:2506.18088*, 2025. URL https://arxiv.org/abs/2506.18088.

Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning Universal Policies via Text-Guided Video Generation. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2302.00111.

Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, et al. LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models. *arXiv preprint arXiv:2510.13626*, 2025. URL https://arxiv.org/abs/2510.13626.

Kristen Grauman, Andrew Westbury, Eugene Byrne, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. URL https://arxiv.org/abs/2110.07058.

Ryan Hoque, Peide Huang, David J. Yoon, Mouli Sivapurapu, and Jian Zhang. EgoDex: Learning Dexterous Manipulation from Large-Scale Egocentric Video. *arXiv preprint arXiv:2505.11709*, 2025. URL https://arxiv.org/abs/2505.11709.

Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is Better to Track Together. In *European Conference on Computer Vision*, 2024. URL https://arxiv.org/abs/2307.07635.

Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, et al. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. *arXiv preprint arXiv:2403.12945*, 2024. URL https://arxiv.org/abs/2403.12945.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, et al. OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*, 2024. URL https://arxiv.org/abs/2406.09246.

Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2601.16163.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.02747.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. *arXiv preprint arXiv:2306.03310*, 2023a. URL https://arxiv.org/abs/2306.03310.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow. In *International Conference on Learning Representations*, 2023b. URL https://arxiv.org/abs/2209.03003.

Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A Large-Scale Simulation Framework for Training and Benchmarking Generalist Robots. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2603.04356.

Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, et al. BridgeData V2: A Dataset for Robot Learning at Scale. *arXiv preprint arXiv:2308.12952*, 2023. URL https://arxiv.org/abs/2308.12952.

Wan Team. Wan: Open and Advanced Large-Scale Video Generative Models. *arXiv preprint arXiv:2503.20314*, 2025. URL https://arxiv.org/abs/2503.20314.

Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point Trajectory Modeling for Policy Learning. In *Robotics: Science and Systems*, 2024. URL https://arxiv.org/abs/2401.00025.

Kun Wu, Chengkai Hou, Jiaming Liu, et al. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. In *Robotics: Science and Systems XXI*, 2025. doi: 10.15607/RSS.2025.XXI.152. URL https://arxiv.org/abs/2412.13877.

Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. VLABench: A Large-Scale Benchmark for Language-Conditioned Robotics Manipulation with Long-Horizon Reasoning Tasks. In *IEEE/CVF International Conference on Computer Vision*, 2025. URL https://arxiv.org/abs/2412.18194.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models Beyond Memorization. *arXiv preprint arXiv:2510.03827*, 2025. URL https://arxiv.org/abs/2510.03827.

Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified World Models: Coupling Video and Action Diffusion for Pretraining on Large Robotic Datasets. *arXiv preprint arXiv:2504.02792*, 2025. URL https://arxiv.org/abs/2504.02792.

Appendix

A Implementation and training details

Prediction horizons and coordinates. The current model predicts 32 action steps in an 80-dimensional typed layout. Consequences use 16 steps with 48 available keypoint slots, each containing two displacement coordinates and one observability coordinate. The agent sequence therefore contains 49 tokens: one state token, 32 action tokens, and 16 consequence tokens. Slots are shared storage positions with source-specific validity masks. The current LIBERO adapter fills 39 consequence slots; EgoDex fills 14 hand-point slots. Unused slots carry a false mask.

Actions use source adapters that preserve the physical meaning of pose, aperture, and joint coordinates. Rotation is represented by the first two columns of its rotation matrix, concatenated in column order, and bypasses statistical scaling. Gripper aperture is first expressed from closed to open in the interval $[0, 1]$ and then mapped to $[-1, 1]$. The active robot recipe uses per-source first and ninety-ninth percentiles for the remaining valid coordinates, with clipping to $[-1, 1]$. Min–max and standard-score normalization are supported alternatives; the archived runs identify the active choice. Displacement uses image-normalized coordinates with gain 4. Observability remains a separate target, including for occluded points whose displacement is masked.

Temporal and visual inputs. A window selects nine frames at a stride of four source steps, spanning 33 time indices. The video autoencoder produces a clean observation latent followed by future latents. The image canvas is 384×256 pixels for the two-view template and 384×320 for the three-view template. The template reserves positions for unavailable views rather than inventing additional observations. DROID and Bridge are sampled at 15 and 5 Hz. The EgoDex training export retains every third frame of the original 30 Hz stream, giving 10 Hz. This corrects the earlier 30 Hz description. The RoboMIND Franka and UR5e exports lack native timestamps; their duration audit retains a 15–20 Hz range. The AgileX and Ego4D adapters are configured at 20 and 30 Hz, respectively. Consequently, a step horizon corresponds to different elapsed durations across sources. Duration-matched comparisons are a separate control for cross-source transfer.

Noise sampling and weighting. For a uniform random variable u and shift r, the implemented level is $\tau = ru/(1 + (r - 1)u)$. The world and action shifts are 5 in the archived runs; the independent consequence draw uses the action shift. The bell-shaped weight is proportional to $\exp[-2(\tau - 1/2)^2] - \exp(-1/2)$, normalized by its mean under the shifted sampling distribution. World, action, and consequence levels are independent during training. The default sampling path decreases all three levels together with 10 Euler steps. Current training uses no additional probability mass on clamped boundary conditions. A boundary-mixture comparison is planned independently of the primary coupling comparison.

Completed adaptation and active pretraining. The completed LIBERO run uses 20,000 optimizer updates, batch 16 per device on four H200 accelerators, one accumulation step, and seed 0. World and agent learning rates both start at 0.0001 with 1,000 warmup updates and cosine decay to 0.01 of the peak. The foundation run uses the settings in Appendix B, with a warmup–stable–decay schedule ending at 0.05 of the peak. Training uses bfloat16 computation, float32 gradient reduction, and activation checkpointing. Foundation training initially uses hybrid sharding across two nodes of four accelerators. The stage resumed at update 8,400 uses full sharding on a single eight-accelerator node, with the same global batch. The video backbone is initialized from released video weights; embodied foundation training is performed within JAM.

The archived training record spans two configuration boundaries. At update 8,400, the run changes to a five-source mixture and activates the displacement filter and pure-noise treatment of missing modalities. At update 10,500, the run switches to the current seven-source mixture in Table 8. The snapshot includes its first logged update, 10,501. Checkpoints are saved every 300 updates. The first retained milestone release is at update 5,100, and the archived log extends to 10,501. The intended downstream launch point is the 15,000-update foundation milestone. These milestones describe training duration rather than completed scaling comparisons. The completed LIBERO adaptation has 401 logged points; its final-window summary averages the last 10% of those points. A zero consequence loss in a robot-only microbatch indicates absent supervision, so foundation summaries condition on the recorded supervised fraction.

B Experimental setup and evaluation

Data and splits. Table 8 gives the current seven-source training mixture, verified against the resolved configuration and runtime sampling probabilities. Sources are sampled by cached training-window counts raised to temperature 0.7. The cache reserves an episode-level holdout using a deterministic one-in-ten partition, with every window from an episode following that assignment. Ego4D groups clips by source video: 436,116 training and 50,856 holdout windows come from 592 and 72 video identifiers, respectively, with zero shared identifiers. The Ego4D adapter supplies world targets; EgoDex additionally supplies projected hand-motion consequences.

Source	Targets	Train windows	Sampling (%)
Robot demonstrations			
DROID	World + action	1,674,648	29.17
BridgeData V2	World + action	454,690	11.71
RoboMIND Franka	World + action	76,446	3.36
RoboMIND UR5e	World + action	439,974	11.44
RoboMIND AgileX	World + action	741,886	16.50
Human video			
EgoDex	World + consequence	738,550	16.45
Ego4D	World	436,116	11.37

Table 8: Current pretraining mixture. Sampling probabilities follow cached training-window counts with temperature 0.7. Counts describe windows rather than unique episodes or duration.

LIBERO, RoboTwin2.0-Full, VLABench, RoboCasa365 target tasks, and RoboDojo use their designated downstream training portions. LIBERO-Plus and LIBERO-PRO supply evaluation only. EBench adds the source-reported mobile manipulation comparison; the JAM row requires the corresponding completed evaluation. Pretraining uses the corpus in Table 8.

Model and optimization. The world stream uses Wan2.2-TI2V-5B through its released Diffusers implementation, initialized with public video weights. The agent stream is trained from scratch. The combined model has approximately 6.02 billion parameters, including a 1.02-billion-parameter agent with width 1,024 and 30 layers. Both streams use 24 attention heads of dimension 128. The video autoencoder and text encoder remain frozen. Foundation training uses eight H200 accelerators, a batch of 16 per device, and four accumulation steps, giving a global batch of 512. AdamW uses world/agent learning rates of 0.00005/0.0001, momentum coefficients 0.9/0.95, weight decay 0.01, and gradient clipping at 1. The planned budget is 60,000 updates with 1,500 warmup updates and a final 6,000-update decay. All three loss weights are 1. The resolved configuration uses independent corruption levels and zero explicit boundary-sampling probability. The completed direct-adaptation run uses four H200 accelerators, global batch 64, and a cosine learning-rate schedule; Appendix A records the remaining adaptation settings.

Tasks and comparison protocol. The core downstream arenas are LIBERO (Liu et al., 2023a), RoboTwin 2.0 Full (Chen et al., 2025), VLABench (Zhang et al., 2025), and the RoboCasa365 target-task split (Nasiriany et al., 2026), with RoboDojo and EBench supplying additional capability tests. Direct adaptation starts from the public video initialization plus a fresh agent; pretrained adaptation starts from a JAM foundation release. The paired comparison fixes the demonstrations, action adapter, training budget, sampler, and environment version. LIBERO-Plus (Fei et al., 2025) and LIBERO-PRO (Zhou et al., 2025) supply robustness tests. RoboTwin clean-to-randomized transfer is evaluated separately from training on its Full mixture. The proposed final protocol uses three training seeds, 50 trials per LIBERO task, and the official one-trial-per-variant LIBERO-Plus enumeration. Each remaining suite follows its pinned task and reset manifest. Full-suite external averages remain distinct from the current task-subset experiments.

Metrics and statistics. Control is measured by task success rate, with subtask progress retained where supplied by the benchmark. World prediction uses perceptual image distance and tracked-motion error. Consequence prediction uses average displacement error in image-normalized coordinates and visibility calibration. Frozen-world probes predict actions and consequences from intermediate world features with generated action and consequence inputs fully corrupted. A shuffled-label probe and an observation-only predictor establish reference levels. Planned uncertainty estimates use a paired hierarchical bootstrap over

training seeds, tasks, and shared episode resets with 10,000 resamples and 95% intervals. Final open-loop comparisons use episode-level resampling. The preliminary probes below use window-level sampling and their separately stated bootstrap protocol.

C Controls required by the scientific question

Separating information and gradients. World-only pretraining supervises future video. Action-only pretraining supervises commands. Independent multitask pretraining supervises both with separate generated-stream computations. Auxiliary supervision reads detached world features. The attached one-way arm uses the same visibility while allowing action gradients through world keys and values. Reciprocal JAM additionally exposes agent keys and values to world queries. Each pretraining arm receives the same source sequence and then the same downstream adaptation procedure. Direct adaptation from public video weights supplies the separate baseline for the value of embodied pretraining.

Strict isolation of coupling requires identical clean conditions in every arm. In the audited implementation, disabling agent visibility also removes the state token from the world stream's attention. The existing visibility switches therefore change both generated-token exchange and part of the conditioning interface. The planned comparison routes equivalent static state and current-observation information to every arm before interpreting a performance difference as an effect of coupling. The paper's target table describes this stricter protocol, whose results remain awaiting measurement. It does not treat the existing switches as an already completed controlled experiment.

Representation probes and interventions. The measured milestone study freezes the world parameters and reads frame-mean features at layers 5, 10, 15, 20, 25, and 29. Initial observations remain clean; future-world inputs use corruption level 0.5. Action and consequence inputs are pure Gaussian noise. Each source contributes up to 4,000 windows from its held-out corpus partition, split into 2,800 probe-training and 1,200 probe-test windows. These internal splits operate on windows and can share episodes. Ridge regularization is selected on a quarter of the probe-training windows and the final readout is refitted on that training split. The grid spans 1 through 100,000. Action R-squared intervals use 1,000 bootstrap resamples over test windows. Figure 5 reports every sampled layer for JAM, the video prior, and a randomly initialized world stream. Its lower panels report the action R-squared difference and the percentage reduction in consequence average displacement error relative to the video prior at each layer. These derived gains are point estimates; the archive provides individual-model action intervals and consequence error estimates. The copy-first consequence reference is plotted alongside all three models. JAM's consequence error falls below this reference at layers 10, 15, and 20. Shuffled-label controls accompany the archived reports. Consequence displacement above magnitude 8 is masked.

This diagnostic tests readout from partially observed future dynamics. Establishing a pretraining effect under matched coupling controls additionally requires episode-disjoint probe fitting, observation-only controls, and matched information at extraction. A capacity-matched nonlinear readout can then separate information accessibility from probe capacity. The current probe scores are preliminary representation measurements with the stated visible future context.

Action shuffling tests sensitivity to the action input. Counterfactual prediction additionally requires a target future obtained by executing the alternative action from the same simulator state. The intervention protocol therefore saves a reset state, executes each admissible command sequence, and scores each generated future against its matching rollout. Corruption tests vary timing and command magnitude within valid control limits. This distinguishes sensitivity from correct prediction of an intervention.

Consequence conditioning compares commands alone, observed consequences alone, and both together on the same test windows. Predicted consequences form a separate condition that retains their model error. The resulting gap measures the cost of prediction and any distribution shift at the conditioning interface. A held-out-embodiment study excludes the target platform from pretraining and fixes its downstream demonstration budget. Camera configuration, temporal duration, and available keypoint groups accompany each result.

Conditional sampling checks. Every conditional use specifies all generated variables. An action-clamped world sampler must continue integrating unobserved consequences. The updated sampler implements

that independent consequence schedule, correcting the earlier coupling to the clamped action level. The archived post-correction test uses the update-5,100 checkpoint, 300 LIBERO holdout windows, and 10 Euler steps. Donor, hold, and time-shuffled actions produce small consequence-error changes whose paired 95% intervals all include zero. Figure 7 uses this corrected report. It measures action-input sensitivity against the observed future, while counterfactual fidelity requires the separately executed futures described above. Conditional benchmark evaluation remains pending. Independent corruption levels define a training family; each deployment path also requires a fidelity check.

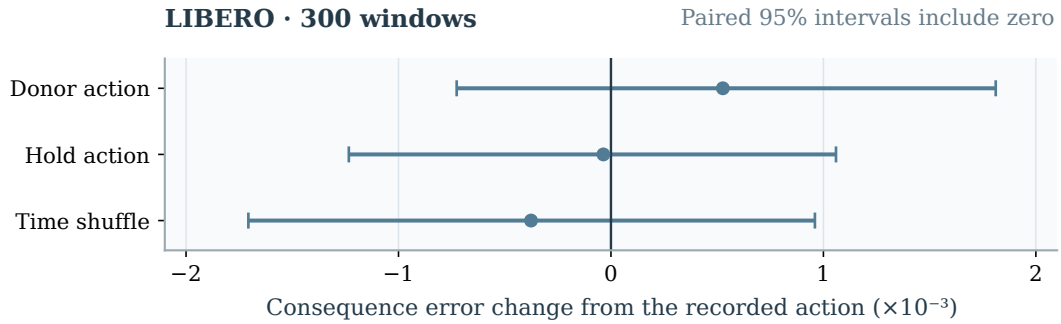

Figure 7: Does changing the action input change consequence error? At update 5,100, donor actions, held actions, and time-shuffled actions are compared with the recorded action on the same 300 LIBERO windows using 10 Euler steps. Positive values indicate higher consequence error. Points show mean changes and bars show paired 95% intervals. All intervals include zero, leaving the direction of each effect unresolved. This input-sensitivity check uses the observed future as its target; intervention fidelity requires separately executed alternative-action futures.

D Evidence boundaries and failure analysis

The evidence package contains the completed adaptation log, its result summary, a fixed prefix of the ongoing foundation log, and sanitized resolved configurations. The public manifest records source paths, file hashes, the audit revision, and the status of each artifact. Local asset paths are replaced by logical references in the published configurations; original-file hashes preserve the connection to the inspected source. These configuration snapshots document the runs and are not standalone launch configurations.

The LIBERO closed-loop evaluation was interrupted before a complete suite result file was produced. Partial rollout videos therefore contribute neither success rates nor selected qualitative evidence to the paper. Foundation transfer and coupling ablations await completed runs. The milestone probes and post-correction sensitivity report are measured diagnostics with the limitations stated above. External reference results retain their own provenance and evaluation scope. The planned tables and scaling figure use design values inherited or extended from the working manuscript; they are layout targets rather than preregistered outcomes or model predictions.

The data audit identified extreme displacement labels in some human-video windows. A displacement mask rejects magnitudes above 8 from the 8,400-update resume onward. The same resume changes the source mixture, maps absent modalities to pure-noise inputs, and changes the sharding topology. The first segment retains the original label distribution. Figure 4 preserves this boundary and its raw losses. Attributing a change to the label filter requires a matched comparison. Camera motion, occlusion, and the gap between observed and predicted consequences remain additional failure settings.

E External reference results

Main results group benchmarks by manipulation setting, retaining all source baseline rows within their respective panels. Table 9 supplies the standard LIBERO suites alongside the robustness tables in the main text. R1 is the external method identified by the immutable source link, distinct from JAM. The website aggregates source-reported evaluations; its presence alone does not establish that all baseline runs share one evaluation implementation. The archive retains original numeric strings, model labels, source revi-

sions, and a cell-level rendering map. The official export contains no LIBERO-PRO table, so Table 2 uses the benchmark maintainers' own leaderboard. Its normalized values are multiplied by 100 exactly. The four displayed suite scores are unweighted means of their reported perturbation rates, rounded to one decimal. Missing environment tests are excluded from these means and flagged on the affected models. Total retains the published aggregate. This summary preserves the difference between derived suite averages and source-reported totals. External protocol scopes remain attached to their values until a matched JAM evaluation is available.

LIBERO					
Model	Spatial	Object	Goal	Long	Mean
OpenVLA	84.7	88.4	79.2	53.7	76.5
π_0	98.0	96.8	94.4	88.4	94.4
StarVLA	97.8	98.6	96.2	93.8	96.6
$\pi_{0.5}$	98.8	98.2	98.0	92.4	96.9
GR00T-N1.6	97.7	98.5	97.5	94.4	97.0
OpenVLA-OFT	97.6	98.4	97.9	94.5	97.1
X-VLA	98.2	98.6	97.8	97.6	98.1
ABot-M0	98.8	99.8	99.0	96.6	98.6
Being-H0.5	99.2	99.6	99.4	97.4	98.9
Qwen-RobotManip	NR	NR	NR	NR	99.2
Fast-WAM	98.2	**100.0**	97.0	95.2	97.6
Motus	96.8	99.8	96.6	97.6	97.7
ImageWAM	97.2	99.2	98.8	98.4	98.4
LingBot-VA	98.5	99.6	97.2	**98.5**	98.5
DiT4DiT	NR	NR	NR	NR	98.6
Being-H0.7	NR	NR	NR	NR	99.2
ABot-M0.5	**100.0**	99.8	99.4	98.4	**99.4**
R1	99.6	99.6	**99.8**	98.2	99.3
JAM	Awaiting evaluation				

Table 9: LIBERO task success. The four suites measure spatial, object, goal, and long-horizon control. Source: external reference R1. Bold marks the highest reported external value per column; NR means unreported. The blue JAM row awaits a completed evaluation.

F Reproduction and experiment ledger

Measured logs, external references, and layout targets occupy separate artifact files. The README gives the complete build procedure. The commands `python3 scripts/build_artifacts.py` and `python3 scripts/validate.py` regenerate the numerical displays and check evidence consistency, respectively.

The protocol ledger links each scientific question to inspected source configurations and required result artifacts. It identifies controls that still require implementation changes. Promoting a target to a measurement requires its result file, checkpoint, evaluation manifest, and aggregation command; the corresponding scientific claim is then reviewed against that evidence.